

1. Escolha e apresentação do caso

1.1 Empresa

Meta Platforms (Instagram)

1.2 Problema

O Instagram precisa recomendar conteúdos relevantes para bilhões de usuários diariamente.

Cada usuário:

- segue pessoas diferentes;
- possui interesses diferentes;
- interage com conteúdos diferentes.

Sem um sistema eficiente, os usuários receberiam conteúdos aleatórios e a retenção da plataforma diminuiria.

1.3 Impacto no negócio

O sistema de recomendação afeta diretamente:

- tempo de uso da plataforma;
- engajamento;
- retenção de usuários;
- receita com anúncios.

2. Raio-X Técnico do Caso

2.1 - Arquitetura e stack tecnológica

2.1.1 - Linguagens utilizadas

A meta utiliza diversas linguagens no contexto empresa, como Python, C++ e Hack (linguagem derivada de PHP criada por eles). Python é utilizado no treinamento de modelos de machine learning, enquanto componentes de alta performance utilizam linguagens compiladas como C++. O Hack é utilizado no backend do instagram para criação de APIs e serviços.

2.1.2 - Banco de dados

A empresa utiliza diversos bancos de dados que, juntos, conseguem suportar bilhões de registros e consultas em tempo real. Um dos principais sistemas utilizados pela Meta é o TAO (The Associations and Objects), uma camada distribuída de armazenamento e consulta desenvolvida pela Meta para armazenar

relacionamentos sociais em larga escala, armazenando objetos (usuários, postagens, páginas) e associações (curtidas, comentários, seguidores e compartilhamentos).

2.1.3 - Sistemas distribuídos

Além do armazenamento distribuído dos dados sociais, a Meta utiliza sistemas distribuídos especializados para recuperação de candidatos (Candidate Retrieval). Nessa etapa, milhares de possíveis conteúdos são recuperados em paralelo a partir de diferentes fontes, como contas seguidas, interesses semelhantes e padrões históricos de interação.

Essa arquitetura distribuída permite reduzir um universo de bilhões de conteúdos para apenas alguns milhares de candidatos relevantes em poucos milissegundos.

2.1.4 - Cloud

A meta utiliza a própria infraestrutura baseada em vários datacenters distribuídos pelo mundo.

2.1.5 - Cache

A meta utiliza sistemas de cache distribuído, como o memcached, para reduzir consultas repetitivas aos bancos de dados e diminuir a latência para entregar uma resposta mais rápida.

2.1.6 - Processamento em tempo real

O sistema de recomendação precisa processar novas informações o tempo todo. Curtidas, comentários e compartilhamentos podem afetar diretamente o conteúdo recomendado para um usuário, o processamento em tempo real ajuda a manter os resultados atualizados e recomendação mais efetiva.

Para lidar com a constante chegada de novos usuários, publicações e interações, a Meta utiliza modelos capazes de gerar representações atualizadas dos dados sem a necessidade de reprocessar todo o grafo social.

Uma das abordagens utilizadas é o GraphSAGE, uma arquitetura de Graph Neural Network (GNN) que aprende características dos usuários e conteúdos a partir de suas conexões e interações. Isso permite incorporar rapidamente novas informações ao sistema de recomendação.

2.2 - Problema computacional envolvido

O Instagram precisava recomendar conteúdo personalizado para cada usuário que utilizava o aplicativo. A maior dificuldade nesse problema é a quantidade de dados que o Instagram precisaria utilizar na recomendação: bilhões de usuários, bilhões de publicações, interações com postagens, entre outros. Uma abordagem ingênua

exigiria comparar cada usuário com uma quantidade enorme de conteúdos, resultando em custos computacionais inviáveis para bilhões de usuários.

Ou seja, a complexidade está na quantidade de comparações que precisam ser feitas para recomendar um conteúdo para um usuário. Outro ponto importante é a escalabilidade, já que o app e o sistema de recomendação precisa continuar funcionando bem, mesmo com a adição de milhões de novos usuários ou posts.

Os dados crescem continuamente no instagram, novos usuários, novos posts, novas curtidas, novos comentários. A cada segundo o instagram tem dados que precisam ser analisados e combinados pelo modelo de recomendação, então é muito importante que exista um método de armazenar, buscar e analisar dados em grande quantidade de forma rápida.

Além de todos esses pontos, o desempenho é um detalhe bem forte nesse problema. Já que mesmo com bilhões de dados, o resultado precisa aparecer rapidamente.

2.3 - Classificação conceitual do problema

O problema se encaixa na classificação de recomendação e otimização, mesmo não sendo tradicionalmente tratado como um problema NP-completo, sua escala torna inviável uma busca exaustiva.

2.4 - Estratégia utilizada pela empresa

A empresa utilizou um conjunto de estratégias computacionais para resolver o problema de recomendação em larga escala.

A primeira grande fase desse pipeline é o Candidate Retrieval (Recuperação de Candidatos), responsável por filtrar o universo de bilhões de publicações existentes na plataforma para apenas alguns milhares de opções potencialmente relevantes para cada usuário, reduzindo drasticamente o custo computacional das etapas posteriores.

Para realizar essa seleção inicial de forma eficiente, a Meta adota uma abordagem híbrida. São utilizadas heurísticas simples e diretas, baseadas em regras lógicas de conexão e interesse.

Após essa seleção inicial, estruturas de grafos são utilizadas para representar usuários, conteúdos e interações. Nesse modelo, usuários, postagens e demais elementos da plataforma são representados como nós, enquanto curtidas, comentários, compartilhamentos e relacionamentos são representados como conexões. Essa abordagem permite identificar conteúdos relevantes sem analisar todo o conjunto de dados disponível.

Para explorar essas relações, a Meta utiliza técnicas de aprendizado em grafos (Graph Neural Networks ou GNNs), como o GraphSAGE. Esses modelos geram representações vetoriais de usuários e conteúdos a partir de suas conexões no grafo social, permitindo identificar interesses semelhantes e padrões de comportamento mesmo entre usuários que nunca interagiram diretamente.

Para tornar essa busca eficiente, o sistema utiliza embeddings e mecanismos de busca vetorial. Tecnologias como o FAISS (Facebook AI Similarity Search) permitem localizar rapidamente conteúdos semelhantes aos interesses do usuário por meio de técnicas de busca aproximada de vizinhos mais próximos (Approximate Nearest Neighbor ou ANN).

Após a recuperação dos candidatos, modelos de inteligência artificial e machine learning estimam a probabilidade de diversas ações do usuário, como curtir, comentar, compartilhar ou assistir completamente um conteúdo. Com base nessas previsões, o sistema calcula a relevância de cada publicação e define sua posição final no feed.

Por fim, o processamento é executado de forma paralela e distribuída entre milhares de servidores. Essa estratégia permite que diferentes etapas da recomendação sejam realizadas simultaneamente, reduzindo a latência e garantindo que os resultados sejam entregues rapidamente mesmo para bilhões de usuários.

3. Proposta Alternativa do Grupo

Uma opção alternativa seria utilizar um sistema híbrido baseado em embeddings e busca vetorial.

Desafios em utilizar os grafos:

Já foram publicados pela Meta alguns desafios sobre utilizar grafos como solução deste determinado problema, dentre estes desafios conhecidos incluem:

Escalabilidade: o grafo cresce continuamente qualquer interação registrada dentro do Instagram, como curtidas e comentários.

Latência: Gerar recomendações rapidamente ao percorrer múltiplos níveis de grafo pode ser caro.

Atualização constante: Novas interações alteram a relevância dos nós ao longo do tempo.

Consumo de memória: armazenar grandes grafos distribuídos exige muitos recursos.

Problema do cold start: usuários ou conteúdos novos possuem poucas conexões no grafo.

Popularity bias: conteúdos muito populares tendem a dominar as recomendações.

Em vez de modelar usuários e conteúdos como vértices conectados por arestas, cada usuário e cada postagem recebe um vetor numérico (embedding) que representa suas características. Essa abordagem reduz a latência, melhora o tratamento de novos conteúdos e facilita a distribuição do processamento, embora perca parte da capacidade de representar explicitamente as conexões sociais entre usuários.

Exemplo:

Usuário:

[esportes, tecnologia, viagens, culinária]

→ [0.9, 0.8, 0.2, 0.1]

Postagem:

[esportes, tecnologia, viagens, culinária]

→ [0.8, 0.7, 0.1, 0.0]

A recomendação é feita calculando similaridade entre vetores (por exemplo, similaridade do cosseno).

Estrutura de Dados

Usuário:

[esportes, tecnologia, viagens, culinária]

→ [0.9, 0.8, 0.2, 0.1]

Postagem:

[esportes, tecnologia, viagens, culinária]

→ [0.8, 0.7, 0.1, 0.0]

- Usuário A → Postagem X
- Usuário A → Usuário B
- Usuário B → Postagem Y

Nesse modelo, recomendações e conexões são obtidas percorrendo as relações existentes no grafo.

Uma alternativa amplamente utilizada em sistemas modernos de recomendação é o emprego de índices vetoriais. Nesse caso, usuários e conteúdos são representados por vetores numéricos de alta dimensão, chamados embeddings. Por exemplo:

- Usuário A → vetor de 512 dimensões
- Postagem X → vetor de 512 dimensões

- Postagem $Y \rightarrow$ vetor de 512 dimensões

A similaridade entre usuários e conteúdos é calculada diretamente no espaço vetorial, permitindo identificar itens semanticamente relacionados sem a necessidade de percorrer explicitamente as conexões de um grafo.

Para realizar buscas eficientes nesses vetores, são utilizadas estruturas especializadas de indexação, entre as quais se destacam:

- HNSW (Hierarchical Navigable Small World)
- FAISS (Facebook AI Similarity Search)
- ScaNN (Scalable Nearest Neighbors)
- Annoy (Approximate Nearest Neighbors Oh Yeah)

Desempenho

Em grafos extensos, a busca por conexões pode exigir percorrer um grande número de vértices e arestas, apresentando complexidade aproximada de: $O(V + E)$, onde: **V** representa o número de vértices; **E** representa o número de arestas.

Por outro lado, algoritmos de busca vetorial aproximada (Approximate Nearest Neighbor ou ANN) conseguem localizar vetores semelhantes com complexidade próxima de: $O(\log n)$, na prática, para grandes coleções indexadas. Como resultado, a latência das recomendações tende a ser significativamente menor.

Escalabilidade

Outra vantagem importante é a escalabilidade. Os índices vetoriais podem ser distribuídos entre múltiplos servidores, permitindo o processamento eficiente de grandes volumes de dados.

Quando uma nova postagem é criada, o sistema realiza apenas três etapas:

1. Gera o embedding da postagem;
2. Adiciona o vetor ao índice;
3. Disponibiliza o conteúdo imediatamente para recomendação.

Dessa forma, não é necessário atualizar grandes regiões de um grafo ou recalcular diversas conexões, tornando o sistema mais simples de manter e expandir.

4. Análise Crítica

Vantagens

- Menor custo computacional.
- Evita percorrer milhões de conexões.
- Melhor tratamento do cold start.

- Mesmo uma postagem nova pode ser recomendada com base em seu conteúdo.
- Atualizações rápidas.
- Inserção simples de novos vetores.
- Fácil de paralelizar.
- Índices vetoriais podem ser distribuídos em vários nós.
- Integração com IA moderna.
- Os embeddings podem ser produzidos por redes neurais que analisam: texto; hashtags; imagens; vídeos; histórico do usuário.

Limitações

- Menor capacidade de capturar relações sociais.
- Grafos representam naturalmente: amizade; seguidores; comunidades; influência. Embeddings podem perder parte dessa informação.
- Necessidade de treinamento.
- É preciso gerar embeddings de qualidade usando modelos de machine learning.
- Possível perda de explicabilidade. No grafo é possível justificar: "Você recebeu esta recomendação porque segue X e Y." Nos embeddings a justificativa é menos intuitiva.

Referências

- <https://about.instagram.com/blog/announcements/how-instagram-recommends-content>
- <https://engineering.fb.com>
- <https://pytorch.org>
- <https://ai.meta.com>
- <https://www.usenix.org/system/files/conference/atc13/atc13-bronson.pdf>
- https://www.usenix.org/system/files/conference/nsdi13/nsdi13-final170_update.pdf
- <https://opensource.fb.com>
- <https://hacklang.org>
- Gpt e Claude como copilotos de pesquisa